

Spring Data JPA

Hands-on 1: Spring Data JPA – Quick Example

application.properties

```
logging.level.org.springframework=info
logging.level.com.cognizant=debug
logging.level.org.hibernate.SQL=trace
logging.level.org.hibernate.type.descriptor.sql=trace
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread %5p %-25.25logger

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

Country.java

```
package com.cognizant.ormlearn.model;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name = "country")
public class Country {

    @Id
    @Column(name = "co_code")
    private String code;

    @Column(name = "co_name")
    private String name;

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

CountryRepository.java

```
package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
}
```

CountryService.java

```
package com.cognizant.ormlearn.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
}
```

OrmLearnApplication.java

```

package com.cognizant.ormlearn;
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;
@SpringBootApplication
public class OrmLearnApplication {
    private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;
    public static void main(String[] args) {
        ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
        LOGGER.info("Inside main");
        countryService = context.getBean(CountryService.class);
        testGetAllCountries();
    }
    private static void testGetAllCountries() {
        LOGGER.info("Start");
        List<Country> countries = countryService.getAllCountries();
        LOGGER.debug("countries={}", countries);
        LOGGER.info("End");
    }
}

```

Output:

```

16-07-26 10:02:16.802 INFO main OrmLearnApplication testGetAllCountries Start
16-07-26 10:02:16.810 TRACE main org.hibernate.SQL select country0_.co_code as
co_code1_0_, country0_.co_name as co_name2_0_ from country country0_
16-07-26 10:02:16.845 DEBUG main OrmLearnApplication testGetAllCountries
countries=[Country [code=IN, name=India], Country [code=US, name=United States of
America]]
16-07-26 10:02:16.846 INFO main OrmLearnApplication testGetAllCountries End

```

Hands-on 2: Hibernate XML Config walk-through

Mapping is defined outside the Java class, in a *.hbm.xml file (<class>, <id>, <property> tying fields to columns).

hibernate.cfg.xml holds connection settings, dialect, and <mapping resource=...> entries.

SessionFactory — heavyweight, thread-safe, built once; factory for Sessions.

Session — lightweight, single-threaded unit of work.

Transaction — abstraction over the JDBC/JTA transaction.

beginTransaction() — starts a transaction on the current session.

commit() — flushes and commits, making changes permanent.

rollback() — reverts changes, usually in a catch block.

session.save() — inserts a transient object, assigns an identifier.

session.createQuery().list() — runs HQL, returns a List.

session.get() — fetches by id, returns null if not found.
session.delete() — removes the persistent object.

Hands-on 3: Hibernate Annotation Config walk-through

Mapping lives directly in the persistence class (e.g. Employee.java) via annotations instead of a separate .hbm.xml.

@Entity — marks the class as persistent.

@Table — maps to a specific table.

@Id — marks the primary key field.

@GeneratedValue — defines how the PK is generated (AUTO, IDENTITY, SEQUENCE).

@Column — maps a field to a column (name, length, nullability).

hibernate.cfg.xml still supplies: Dialect (SQL flavor), Driver (JDBC class), Connection URL, Username/Password.

Hands-on 4: JPA vs Hibernate vs Spring Data JPA

JPA — spec (JSR 338) only, no implementation.

Hibernate — an ORM framework that implements JPA, plus its own extras (caching etc).

Spring Data JPA — sits above a JPA provider like Hibernate; generates repository implementations at runtime, removes boilerplate, integrates @Transactional

```
// Hibernate — manual session/transaction handling
public Integer addEmployee(Employee employee) {
    Session session = factory.openSession();
    Transaction tx = null;
    Integer employeeID = null;
    try {
        tx = session.beginTransaction();
        employeeID = (Integer) session.save(employee);
        tx.commit();
    } catch (HibernateException e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
    return employeeID;
}
```

```
// Spring Data JPA
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {}

@Autowired
private EmployeeRepository employeeRepository;

@Transactional
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}
```

Spring Data JPA removes the manual session/transaction try-catch-finally boilerplate — handled by the container via `@Transactional`.

Hands-on 5: ddl-auto & data population

create — drops & recreates tables every startup.

validate — only checks schema matches, throws if not (used here).

update — adds missing tables/columns, keeps data.

create-drop — creates on startup, drops on shutdown.

```
spring.jpa.hibernate.ddl-auto=validate
```

Deleted existing rows and loaded the full 245-row country list via the provided SQL script.

```
mysql> select count(*) from country;
+-----+
| count(*) |
+-----+
|      245 |
+-----+
```

Hands-on 6: Find country by code

```
package com.cognizant.ormlearn.service.exception;

public class CountryNotFoundException extends Exception {
    private static final long serialVersionUID = 1L;
    public CountryNotFoundException(String message) { super(message); }
}
```

CountryService addition

```
@Transactional
public Country findCountryByCode(String countryCode) throws CountryNotFoundException {
    Optional<Country> result = countryRepository.findById(countryCode);
    if (!result.isPresent()) {
        throw new CountryNotFoundException("Country not found for code: " + countryCode);
    }
    return result.get();
}
```

Test method

```
private static void getAllCountriesTest() throws CountryNotFoundException {
    LOGGER.info("Start");
    Country country = countryService.findCountryByCode("IN");
    LOGGER.debug("Country:{}", country);
    LOGGER.info("End");
}
```

Output:

```
16-07-26 10:15:02.114 INFO main OrmLearnApplication getAllCountriesTest Start
16-07-26 10:15:02.121 TRACE main org.hibernate.SQL select ... from country
country0_ where country0_.co_code=?
16-07-26 10:15:02.129 DEBUG main OrmLearnApplication getAllCountriesTest
Country:Country [code=IN, name=India]
16-07-26 10:15:02.130 INFO main OrmLearnApplication getAllCountriesTest End
```

Hands-on 7: Add a new country

```
@Transactional
public void addCountry(Country country) {
    countryRepository.save(country);
}
```

```
private static void testAddCountry() throws CountryNotFoundException {
    LOGGER.info("Start");
    Country newCountry = new Country();
    newCountry.setCode("XX");
    newCountry.setName("Testland");
    countryService.addCountry(newCountry);
    Country savedCountry = countryService.findCountryByCode("XX");
    LOGGER.debug("Saved Country:{}", savedCountry);
    LOGGER.info("End");
}
```

Output:

```
16-07-26 10:21:40.315 TRACE main org.hibernate.SQL insert into country (co_name,
co_code) values (?, ?)
16-07-26 10:21:40.330 DEBUG main OrmLearnApplication testAddCountry Saved
Country:Country [code=XX, name=Testland]
```

Hands-on 8: Update a country

```
@Transactional
public void updateCountry(String code, String name) throws CountryNotFoundException {
    Optional<Country> result = countryRepository.findById(code);
    if (!result.isPresent()) {
        throw new CountryNotFoundException("Country not found for code: " + code);
    }
    Country country = result.get();
    country.setName(name);
    countryRepository.save(country);
}

private static void testUpdateCountry() throws CountryNotFoundException {
    LOGGER.info("Start");
    countryService.updateCountry("XX", "Testland Updated");
    Country updatedCountry = countryService.findCountryByCode("XX");
    LOGGER.debug("Updated Country:{}", updatedCountry);
    LOGGER.info("End");
}
```

Output:

```
16-07-26 10:27:12.024 TRACE main org.hibernate.SQL    update country set co_name=?
where co_code=?
16-07-26 10:27:12.033 DEBUG main OrmLearnApplication testUpdateCountry Updated
Country:Country [code=XX, name=Testland Updated]
```

Hands-on 9: Delete a country

```
@Transactional
public void deleteCountry(String code) {
    countryRepository.deleteById(code);
}

private static void testDeleteCountry() {
    LOGGER.info("Start");
    countryService.deleteCountry("XX");
    LOGGER.info("End");
}
```

Output:

```
16-07-26 10:33:55.514 TRACE main org.hibernate.SQL    delete from country where
co_code=?
16-07-26 10:33:55.521  INFO main OrmLearnApplication testDeleteCountry End
```